

Session 09 — Study Guide: DTO Inheritance and Closing the Validation Gap

Where the session ends

Both `CreateProjectDto / CreateWorkspaceDto` trim `name / key / slug` with `@Transform()` before validating them, and `UpdateProjectDto / UpdateWorkspaceDto` no longer have any decorators of their own — they inherit everything from their own Create DTO via `extends PartialType(...)`. A real, confirmed bug — whitespace-only names silently accepted by `POST` — is closed, for good, in a way that cannot silently reappear in only one of the two DTOs again.

The real gap this session closes

`@IsNotEmpty()` only checks that a string's length is greater than zero. A string of three spaces, `" "`, has length 3 — it passes. `POST /projects` and `POST /workspaces` currently accept and store a name like this; `PATCH` on the same field already correctly rejects it, because a fix was added to the Update DTOs but never the Create DTOs. This session closes that gap — twice: once the quick way, then properly.

Step one: `@Transform()` — rewrite before you check

`class-transformer`'s `@Transform()` decorator runs BEFORE `class-validator`'s own checks, as part of building the real class instance the pipe validates against:

TS

```
@IsString()
@Transform(({ value }) => value.trim())
@IsNotEmpty()
name: string;
```

`(({ value }) => value.trim())` receives one options object (not the raw value) and returns the trimmed string. By the time `@IsNotEmpty()` runs, `" "` has already become `""` — correctly rejected.

Step two: the cost of copying the fix by hand

Adding the same three lines to BOTH `CreateProjectDto` and `UpdateProjectDto` works — but now the exact same rule exists in two places. That is not just untidy: it is *why* the original bug happened. A fix was made to one DTO and the other was forgotten. Two classes, one shape, is a real risk every time either one changes.

Step three: `PartialType` — one source of truth

`@nestjs/mapped-types` provides `PartialType()`, which builds a new DTO class from an existing one's own fields and decorators — including `@Transform()`:

```
import { PartialType } from '@nestjs/mapped-types';
import { CreateProjectDto } from './create-project.dto';

export class UpdateProjectDto extends PartialType(CreateProjectDto) {}
```

Plain `extends CreateProjectDto` inherits the parent DTO's behavior — `class-validator`'s own decorator metadata IS inherited through ordinary class inheritance — but it does not turn the required Create fields into optional Update fields; they stay required. `PartialType(CreateProjectDto)` reads `CreateProjectDto`'s own `class-validator` / `class-transformer` decorator metadata and builds a new class with that same set of rules, with `@IsOptional()` layered on top of every field — the same rules, but now every field is optional, which is exactly what PATCH/update needs.

Now there is exactly one place that defines what a valid `name` / `key` looks like. A future rule change to `CreateProjectDto` reaches `UpdateProjectDto` automatically.

PartialType's siblings (FLEX — not built today)

`@nestjs/mapped-types` also has `PickType(Dto, ['field'])` (only the named fields) and `OmitType(Dto, ['field'])` (every field except the named ones) — same idea, different shaping. Not used in B4F Board yet; worth knowing the names exist.

Common beginner mistakes

- `@Transform(value => value.trim())` instead of `@Transform(({ value }) => value.trim())`. The callback receives one options object, not the raw value.
- **Forgetting the `class-transformer` / `@nestjs/mapped-types` import in a new file.** Neither is shared automatically — every DTO file that uses them needs its own import line.
- **Fixing the Update DTO first, out of habit.** Once `PartialType` is in play, the fix always belongs on the Create DTO — Update inherits it for free.
- **Leaving decorators behind "just in case" inside a `PartialType`-derived class.** If nothing is different, the class body should be empty: `{}`.
- **Forgetting the empty braces entirely** (`extends PartialType(CreateProjectDto)` with no `{}`) — a TypeScript class always needs a body.

Not yet

These exist in NestJS, but are deliberately **not** covered this session — do not use them yet, and do not worry if you have seen the names before:

- `PickType` / `OmitType` hands-on (discussion only this session)
- Pipes that validate route parameters (a different pipe mechanism than today's)
- Guards, interceptors, custom exception filters
- Provider tokens, custom providers, injection scopes
- Custom decorators (`createParamDecorator`)

- `@nestjs/config` / `.env` configuration
- Swagger/OpenAPI
- Any database, authentication, or Next.js

Glossary

- `@Transform()` — a `class-transformer` decorator that rewrites a field's value before validation runs.
- `PartialType(Dto)` — builds a new DTO class from an existing one, same fields/decorators, every field optional.
- **Decorator metadata** — information `reflect-metadata` stores about a class's own decorators, readable (and copyable) at runtime — this is what makes `PartialType` possible.
- **One source of truth** — a value or rule defined in exactly one place, so every use of it stays consistent automatically.

Review questions

1. Why does `@Transform()` run before `@IsNotEmpty()`, regardless of which order they're written in the file?
2. What goes wrong if a teammate writes `class UpdateWorkspaceDto extends CreateWorkspaceDto {}` (plain `extends`, no `PartialType`)?
3. Which DTO — Create or Update — should a new validation rule always be added to, once `PartialType` is in use? Why?

Answers

1. `class-transformer`'s transformation phase always runs before `class-validator`'s checking phase inside `ValidationPipe` — this is inherent to the pipeline, not controlled by decorator order in the source file.
2. The inherited Create fields remain required, so it behaves like the Create DTO rather than a partial update DTO — plain `extends` inherits `CreateWorkspaceDto`'s own `class-validator` / `class-transformer` decorators (ordinary class inheritance does carry decorator metadata), but it does not make any field optional. `PartialType()` keeps the same rules but makes every field optional, which is what an Update DTO actually needs.
3. Create — because Update is derived FROM Create via `PartialType()`. A rule added only to Update would not exist on Create at all.

What you should be able to explain now

- Why `@Transform()` must run before the validation checks it supports.
- Why duplicating the same decorators across two DTOs is a real, not just cosmetic, risk.
- What `PartialType()` actually does at runtime, and why it is different from plain `extends`.
- The real bug this session closed, and which HTTP method had it before today.